

Prozesse

Key Takeaways

- Prozesse sind laufende Programme
- Bestehen aus eigenem Speicherbereich, Instruction Pointer, Register und Stack
- Werden auf CPU ausgeführt (einer auf einmal (pro Kern))

Prozesse sind ausgeführte (also derzeit laufende) Programme. Prozesse, zusammen mit ihren Daten und Speicherbereichen, werden in die CPU geladen um dort ausgeführt zu werden.

Eine CPU kann nur einen Prozess gleichzeitig ausführen.

Heutzutage haben PCs mehrere **Kerne**, also mehrere Prozessoren in einer CPU. Diese erlauben das wahre parallele Ausführen von Programmen. Geht man jedoch von der minimalen Menge an Kernen (1) aus, muss es trotzdem möglich sein, mehrere Prozesse zur selben Zeit zu verwenden.

Dafür gibt es das Konzept der **Quasiparallelität**, welche vom OS verwaltet wird. Die Hauptfrage dieses Themengebietes ist:

Wer kommt wann und wie lange dran?

Dieses Verwalten, bzw die Techniken und Logiken dahinter, werden als Scheduling bezeichnet.

Bestandteile eines Prozesses

Ein Prozess besteht aus verschiedenen Teilen, welche in die CPU geladen werden müssen:

- Code / Befehlszähler (Instruction Pointer)
 - Pointer bzw. Zähler, welcher immer auf die aktuell auszuführende Stelle im Programm zeigt. Wichtig um nach einem Context Switch wieder an der richtigen Stelle einzusteigen
- Register
 - Siehe EIR
- Variablen
 - Daten, welche der Prozess braucht um korrekt ausgeführt werden zu können

Der Prozessor

Key Takeaways

- CPU Teil, welcher Prozesse ausführt
- Lädt die Speicher und Pointer, führt Befehle aus
- Bekommt vom Scheduler gesagt, wann er welchen Prozess laden soll

Ein Prozessor ist der zentrale Teil der CPU. Er ermöglicht das Ausführen von Prozessen nachdem die korrekten Daten geladen wurden. Welche Daten wann für wie lange geladen werden obliegt dem Betriebssystem.

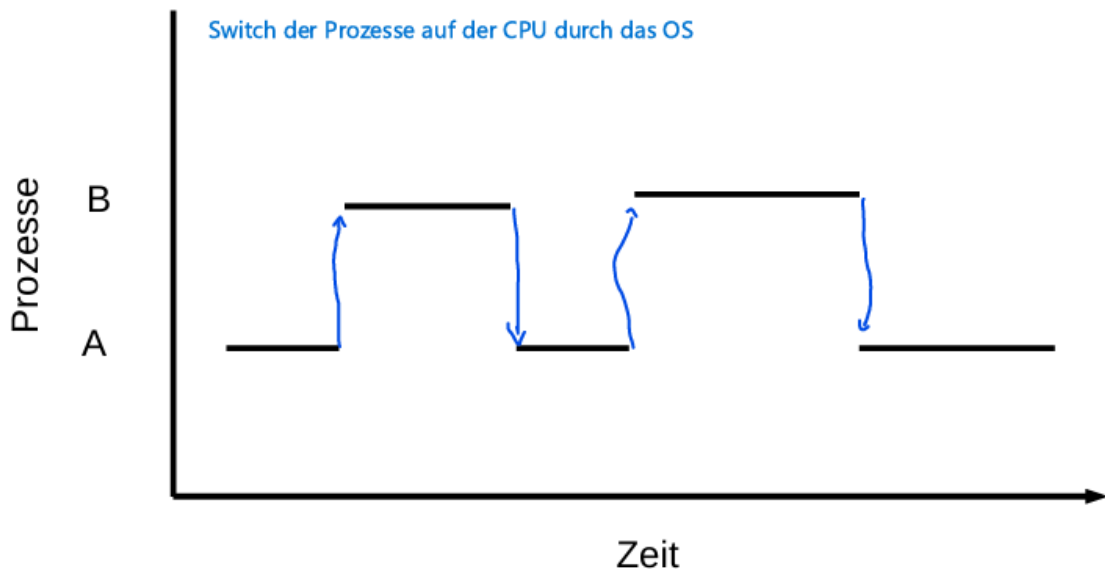
(Für gewöhnlich) belässt das OS einen Prozess nur eine **bestimmte Zeit** im Prozessor und tauscht ihn dann mit einem anderen Prozess. Das geschieht auch, wenn der vorherige Prozess noch nicht zu Ende bearbeitet war. Zu einem Zeitpunkt wird immer nur ein Prozess ausgeführt.

Quasiparallelität

Key Takeaways

- Quasi - da es nur so wirkt als wäre es parallel
- In Wahrheit sequenziell mit schnellem Wechsel der einzelnen Prozesse (Context Switch)
- Wichtig für Response Time des Systems und Lag-freies Erlebnis für den User

Quasiparallelität beschreibt das schnelle Wechseln von Prozessen in einem einzigen Prozessor. Jeder Prozess läuft nur eine kurze Zeit, bevor das Betriebssystem die derzeitige Position und alle Daten zwischenspeichert und einen anderen Prozess lädt. Durch die hohen Geschwindigkeiten entsteht für den Mensch die Illusion, mehrere Programme würden gleichzeitig ausgeführt.



Hierbei ist zu beachten, dass natürlich für jeden Wechsel (auch **Context Switch** genannt) eine gewisse Zeit gebraucht wird, da ja alle Daten zwischengespeichert und neue Daten geladen werden müssen. Daher ist die Balance zwischen zu vielen Context Switches und zu langen Prozesszeiten eines einzigen Prozesses der Schlüssel zu einer sich gut anführenden Quasiparallelität.

Erstellen von Prozessen

Key Takeaways

- Prozesse werden immer von anderen Prozessen erstellt
- Prozesse sind Kopien ihrer Eltern mit anderer IP, deren Speicher dann mit neuem Code überladen wird.
- Prozesse werden vom OS über System Calls angefordert.

Ein Prozess kann auf verschiedene Weisen entstehen:

- OS-Initialisierung
 - Diese Prozesse sind die ersten Prozesse welche nach dem Start laufen und das System auf den Betrieb vorbereiten. Laufen danach normalerweise nicht mehr.
- System Calls
 - Durch System Calls (oder andere Interrupts) werden ISR aufgerufen, welche ebenfalls Prozesse sind die ausgeführt werden müssen. Haben je nach Interrupt eine sehr hohe Priorität. System Calls sind die häufigste Ursache für die Prozesserstellung.
- Durch den Benutzer

- Die bekannteste Art einen Prozess zu starten, zB durch Doppelklicken auf eine .exe Datei.
- Iwas mit Stabeljobs (steht in den Folien, is aus den 60ern, einfach älter als mein Dad smh)

Erstellungsablauf (Linux)

Die Erstellung eines Prozesses wird durch einen **Elternprozess** initialisiert. Ein neuer Prozess (**Kindprozess**) hat also immer einen Prozess, welcher ihn erstellt hat. Das gilt für alle Prozesse außer den allerersten Prozess.

Der Elternprozess erstellt eine exakte Kopie von sich selbst, setzt jedoch die ProzessID auf 0. Durch diese ID können Eltern und Kind auseinandergehalten werden. Danach wird der Adressraum des Kindprozesses mit dem gewünschten Programm überschrieben. Das bedeutet, die Inhalte des Elternprozesses sind nicht mehr im Kindprozess, sondern nur noch die neuen Programminhalte. Diese werden dann ausgeführt.

Ist der Kindprozess fertig, beendet er sich und der Elternprozess kommt wieder dran (Funktional so wie ein Funktionsaufruf innerhalb einer anderen Funktion)

Über diese Methode hat man früher zwischen Prozessen gewechselt, bevor es **Threads** gab.

Beenden von Prozessen

Key Takeaways

- Prozesse können geregelt durch Erreichen ihres Endes beendet werden
- Können von außen gezwungen werden (Task Manager)
- Können durch Fehler abstürzen

So wie ein Prozess durch verschiedene Geschehnisse entstehen kann, kann er auch durch verschiedene Geschehnisse beendet werden.

- Freiwilliges Beenden (selbstständiges Beenden)
 - Wie der Name schon sagt, beendet sich der Prozess zB durch Aufruf von `exit(0)` (Linux) / `ExitProcess()` (Windows), freiwillig selbst, weil er fertig ist.
- Beenden aufgrund eines Fehlers (freiwillig)
 - Der Prozess erkennt einen Fehler und beendet sich als Reaktion selbst mit einem Fehlercode zB `exit(-1)`
- Beenden aufgrund eines schweren Fehlers (unfreiwillig)
 - Der Prozess läuft in einen unbehandelten Fehler und stürzt ab. (Division 0, Speicheradressenfehler, allg. Programmierfehler)
- Beenden von außen durch anderen Prozess

- Task-Manager und verwandte Prozesse können über Systemcalls andere Prozesse beenden

Prozesszustände

Key Takeaways

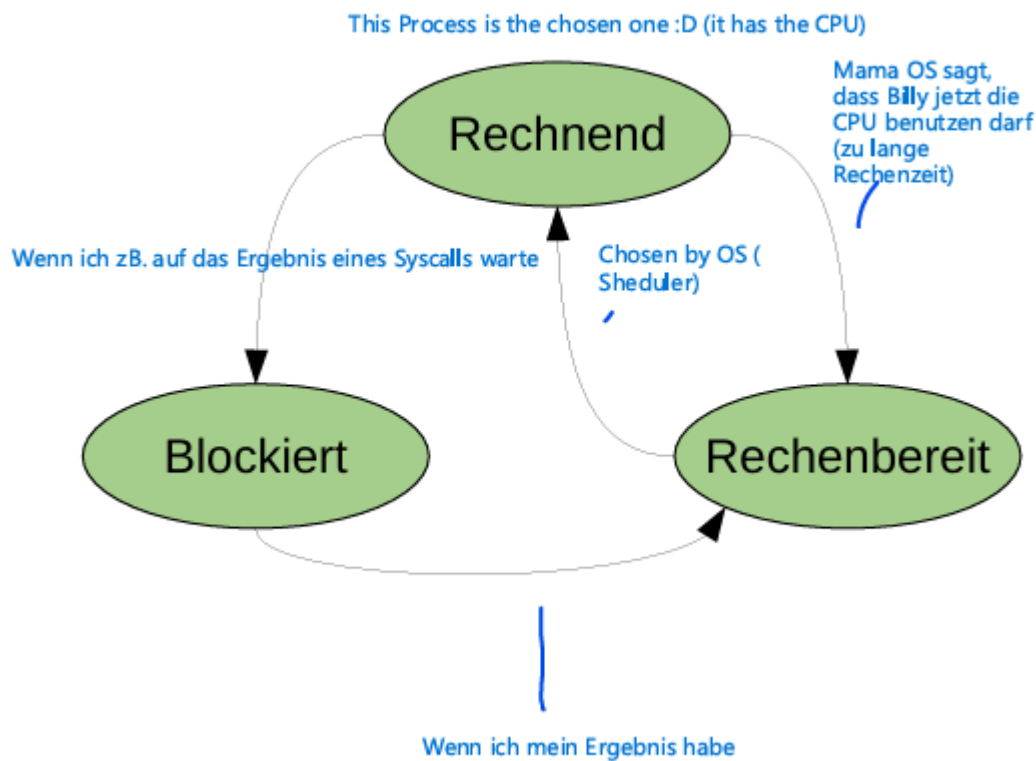
- Prozesse haben Zustände damit OS weiß, welcher wann geladen werden kann
 - blockierend: Wartet auf Rückmeldung eines anderen Systems
 - Rechenbereit: Könnte auf die CPU geladen werden
 - Rechnend: Läuft derzeit auf der CPU

Ein Prozess muss oft mit dem Betriebssystem, der Hardware oder anderen Prozessen interagieren. Möchte ein Programm ein File vom Speicher lesen, muss erst ein Befehl an das OS gehen, welches dann das angegebene File im Dateisystem suchen und liefern muss (sofern es gefunden wird)

In dieser Zeit kann der Prozess nicht weiterarbeiten und würde die CPU komplett blockieren, was sehr ineffizient ist. Daher wird er meist vom OS von der CPU gewechselt. Damit das OS allerdings weiß, welche Prozesse auf etwas warten und welche sofort bereit sind, gibt es

Prozesszustände

- Blockiert
 - Ist ein Prozess "blockiert", kann er nicht weiterarbeiten bis er eine bestimmte Information vom OS oder einem anderen Prozess erhalten hat.
- Rechenbereit
 - Prozesse welche sofort (weiter)arbeiten könnten, aber noch in die CPU geladen werden müssen, sind rechenbereit
- Rechnend
 - The Chosen One, der Prozess welcher zu diesem Zeitpunkt die CPU verwendet und aktiv arbeitet.



Prozesse wechseln - Prozesstabelle

Key Takeaways

- Prozesstabelle hält alle Prozesse, ihre Dateien und Speicher
- Braucht das OS um zu wissen, wo der Prozess ist, welche es gibt und was dazugehört
- Informationen werden zum Context Switch verwendet.

Nun wissen wir, in welchem Zustand sich die Prozesse befinden, aber wo sie sind, wie viele es sind und was sie für Daten brauchen ist auch eine recht wichtige Information. Dafür gibt es die **Prozesstabelle**, wie der Name vermuten lässt, eine Tabelle, in welcher alle derzeit aktiven Prozesse abgelegt werden (bzw. ihre Position im Speicher)

Weiters wird gespeichert, was jeder Prozess für Dateien benötigt und wo in der Prozessauführung man sich derzeit befindet.

Prozessverwaltung	Speicherverwaltung	Dateiverwaltung
Register	Zeiger auf Textsegment	Arbeitsverzeichnis
Befehlszähler	Zeiger auf Datensegment	Dateideskriptor
Stackpointer	Zeiger auf Stacksegment	User Id
Programmstatuswort	...	Gruppen Id
Prozess ID (PID)		...
Benutzte CPU Zeit		
Prozesszustand		

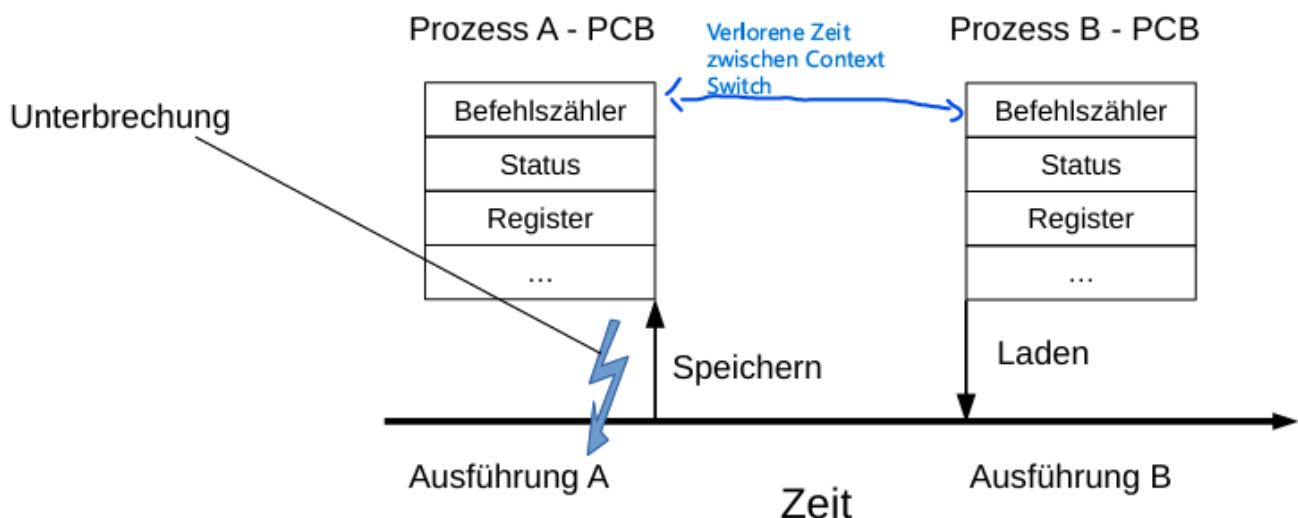
Daten, was ist der aktuelle Stand des Prozesses?
(Wichtig, damit ich den Prozess wieder starten kann)

Daten die mir sagen, wo genau der Code ist, den ich ausführen will

Daten die mir sagen, welche Files offen waren, welche Rechte der Prozess hat (sudo) usw.

Mithilfe dieser Tabelle hat das OS alle nötigen Informationen, um einen Prozess auszuwechseln. Dieser Wechsel wird als **Context Switch** bezeichnet, da eben der gesamte Kontext auf der CPU gewechselt wird.

- Zuerst wird der laufende Prozess angehalten. Durch den Instruction Pointer weiß das OS weiterhin, wo der Prozess stehengeblieben ist.
- Alle Daten des aktuellen Prozesses werden gespeichert und die nötigen Referenzen in der Prozesstabelle abgelegt.
- Mithilfe der Prozesstabelle wird ein neuer Prozess ausgewählt
- Die Daten des neuen Prozesses werden geladen, der Instruction Pointer gesetzt und der Prozess gestartet.



Threads

Key Takeaways

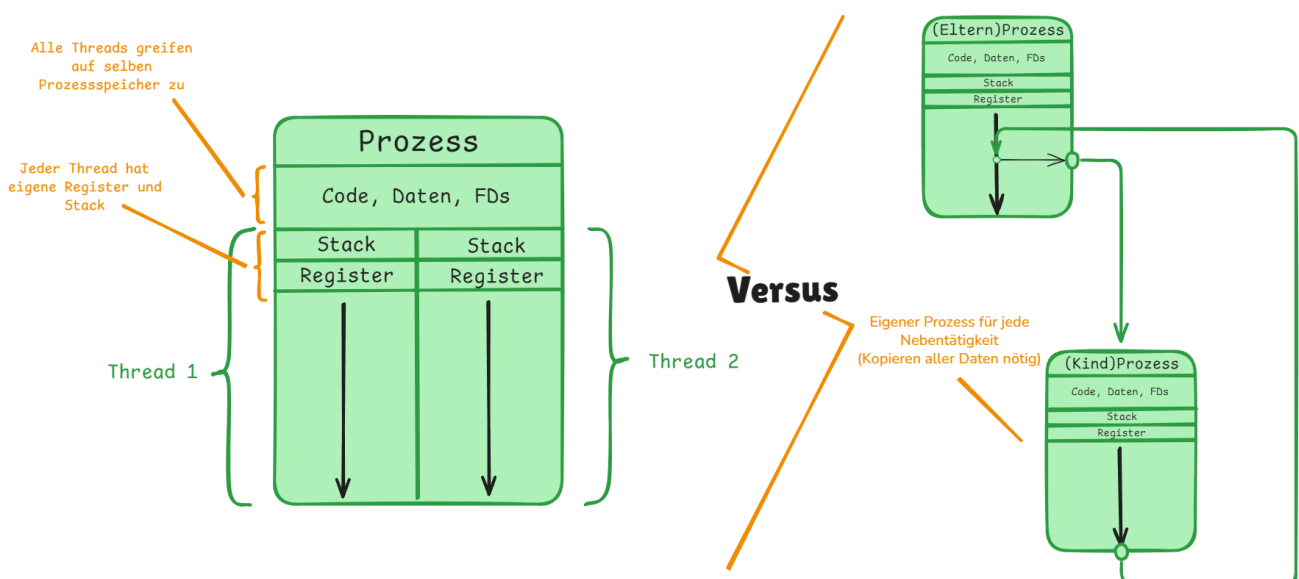
- Threads sind Alternative zu Kindsprozessen (Prozess im Prozess)
- Liegen innerhalb eines Prozesses teilen sich gleichen Speicher
- Sehr light weight, ein Prozess kann beliebig viele Threads haben

- Durch das Speicher teilen anfällig für Fehler in Dateikonsistenz

Früher hat man um einen Unterprozess in einem Prozess zu realisieren einen weiteren, vollwertigen Prozess gestartet. Das ist aufwendig und man muss Daten zwischen den verschiedenen Speicheradressen der Prozesse hin und her speichern.

Threads lösen dieses Problem. Sie sind quasi ein "Prozess im Prozess". Ein Prozess kann mehrere Threads parallel starten. Hier sind einige wichtige Punkte zu Threads:

- Threads benutzen den gleichen Speicher!
 - Anders als eigenständige Prozesse greifen alle Threads eines Prozesses auf den gleichen Speicher (eben den Speicher des Prozesses) zu. Das erspart Hin- und Herspeichern zwischen Speicherbereichen und erleichtert die Zusammenarbeit, eröffnet aber auch neue Möglichkeiten für Dateninkonsistenzen und andere Fehler (siehe Race-Conditions, Semaphore)
- Threads sind leichtgewichtiger
 - Da sie fester Teil eines Prozesses sind und keine eigenen Speicher haben, sind sie schneller und leichtgewichtiger als Kindprozesse.
- Threads sind leistungsfähig
 - Threads erlauben es Prozessen weiterzuarbeiten, selbst wenn ein Programmteil eigentlich blockiert ist. zB. Kann das File Handling in einen eigenen Thread verlagert werden, während ein anderer Thread weiterrechnet.



Implementierungen

Es gibt verschiedene Arten Threads real zu implementieren. Die 3 Hauptarten unterscheiden sich dadurch, in welchem Space die Threads liegen, und wer für sie verantwortlich ist.

Implementierung im OS (Kernel Space)

In dieser Implementierung hält das OS, ähnlich wie bei Prozessen selbst, alle wichtigen Informationen der Threads in einer Threadtabelle fest. Dort werden Daten wie der Speicherort des Stacks, der Register, der Instruction Pointer und der derzeitige Threadzustand abgelegt. Die Logik ist exakt die selbe wie bei Prozessen.

Implementierung im User Space

In dieser Implementation wird innerhalb eines Prozesses eine eigens implementierte Thread Tabelle und Scheduler verwendet. Diese erstellen auf dem User Level "Threads", die dann alle im selben Prozess laufen. Diese Implementierung kommt ohne System Calls aus und erlaubt schnelle Context Switches ohne Speicherung der Daten. Allerdings ist auf OS-Ebene alles Bestandteil des selben Prozesses und läuft auf OS Ebene auch alles im selben Prozessstrang. Wenn also einer dieser Threads "blockiert", kann es passieren dass alle Threads auf diesen warten müssen.

Hybride Implementierung

Verwendet beide oberen Konzepte gemeinsam.

Windows verwendet hybride Implementierung. Linux verwendet quasi keine Threads, sondern nutzt nach wie vor eigene Kindprozesse, nur mit kleinen Unterschieden. Die Vorteile von Threads werden durch Permissions nachgebaut anstatt sie eigens zu implementieren (kinda weird bro but ok)

Vor / Nachteile verschiedener "Parallelitäten"

Key Takeaways

- Prinzipiell: Parallel = Schnell aber hardwareabhängig und teuer
- Quasiparallel: Flexibel aber schwierig gut zu machen und gefährdet für Fehler
- Sequenziell: Simpel und stabil aber langsam und unresponsive

Sequenzielle Prozessabarbeitung (Einer nach dem anderen)

- • Simpel, die einfachste Art, Prozesse abzuarbeiten
- • Context Switch nur nach Ende eines Prozesses - wenig Zeit geht verloren
- • Fairness - jeder Prozess bekommt immer genau so viel CPU Zeit wie er braucht
- • Berechenbar - Die Zeit, bis ein Prozess ausgeführt werden kann, kann sehr gut vorausberechnet werden
- • Niedrige Reaktanz - Kein Unterbrechen der Prozesse macht Eingaben stark verzögert
- • Langsam - Blockierende Prozesse werden nicht ausgewechselt, neue Prozesse müssen warten bis alle vorherigen Prozesse fertig ausgeführt wurden.

- • Nutzerunfreundlich - Keine Quasiparallelität sorgt dafür, dass immer genau eine Sache auf einmal passieren kann. Während ein Prozess lädt kann kein anderes Programm gestartet, geschweige denn die Maus bewegt werden.

Mehrere Prozesse parallel (mehrere Kerne)

- • Parallel, mehrere Dinge passieren real gleichzeitig, was die Geschwindigkeit drastisch erhöht
- • Lasten können zwischen mehreren Kernen aufgeteilt werden
- • Es kommt nicht / viel seltener zu Blockaden, da Eingaben von anderen Kernen abgefangen werden können
- • Durch reale Parallelität kaum Context-Switches nötig
- • Teurer, da mehr Kerne benötigt werden
- • Hardwareabhängig, Scheduling für diese Maschinen funktioniert nur auf More-Core Systems
- • Kommunikation zwischen Kernen ist eine Schwachstelle, wie auch bei der Quasiparallelität, kann aber umgangen werden sofern Kerne immer alle für ihren Prozess relevanten Nebenprozesse zugewiesen bekommen.

Ein Prozess, mehrere Threads

- • Flexibel, benötigt keine spezifische Hardware und erlaubt mehrere Prozesse "gleichzeitig"
- • Im Vergleich zu Kindsprozessen leichtgewichtiger und billiger.
- • Erlaubt das Starten vieler Threads und das logische Aufteilen von Programmteilen innerhalb eines Prozesses
- • Für jeden Threadwechsel ist ein Switch nötig, auch wenn hier die Speicherdaten nicht extra neu geladen werden müssen, sind zumindest die Register und der Stack zu überspeichern
- • Durch den Zugriff auf den selben Speicherbereich können Schwierigkeiten bei der Dateikonsistenz (Race-Conditions) entstehen.

1 Prozess, kein I/O Blocking - High Priority Interrupts

- • Selbe Vorteile und Probleme wie bei der Sequenziellen Abarbeitung, nur dass zumindest Eingaben nicht so lange blockiert werden. Dafür braucht es dann aber mehr Context-Switches.

Scheduling

- Bezeichnet das Vorgehen des Schedulers, zu entscheiden welcher Prozess wann dran kommt
- Verfolgt Ziele
 - Fairness
 - Balance
 - Response Time
 - Policy Enforcement

Scheduling bezeichnet den Vorgang des OS bei welchem entschieden wird, welcher Prozess zu welchem Zeitpunkt für wie lange auf der CPU ausgeführt werden soll. Dies wird vom Betriebssystem anhand verschiedener Metriken berechnet. Wird durch diese Berechnung ein neuer Prozess ausgewählt und in die CPU geladen, wird dies als **Context Switch** bezeichnet.

Ziele des Scheduling

Um ein interaktives System funktional und effizient zu halten und das User-Feeling gut zu halten, muss der Scheduler einige Prinzipien beachten.

Fairness

Prozesse, welche sich in Wichtigkeit ähneln, sollten ca. gleich oft drankommen.

Balance

Es sollte eine Balance zwischen der CPU und I/O Auslastung vorliegen, um Überforderung eines der beiden Systeme zu vermeiden. Weiters sollten sie möglichst effizient genutzt werden. Wird zB eine Anfrage an die Hardware geschickt, kann die CPU bis zur Antwort andere Dinge berechnen anstatt auf die Antwort zu warten und damit Zeit zu verschwenden.

Response Time

Die Zeit zwischen einer Nutzereingabe und Reaktion sollte so gering wie möglich sein (ansonsten LAG) Dementsprechend genießen System Calls eine hohe Priorität im Scheduling-Prozess.

Policy Enforcement

Gibt es im Betriebssystem die Möglichkeit, das Verhalten des Schedulers einzustellen, müssen diese Einstellungen auch respektiert werden.

Context Switches

- Vorgang einen neuen Prozess auf die CPU zu laden
- Braucht Zeit, da Speicher überschrieben werden muss
- Erzeugt Overhead, ermöglicht aber Quasiparallelität
- Einteilung von Prozessen erleichtert Entscheidung für Switch:
 - IO Bound: Braucht viel IO, wenig CPU, schnell fertig, deshalb oft hochprior
 - CPU Bound: Braucht viel CPU, wenig IO, langsamer, deshalb niederprior
- Context Switches passieren (abhängig von algorithmus):
 - Nach Enden des aktuellen Prozesses
 - Bei Interrupt
 - Bei Blockieren des aktuellen Prozesses

Wie bereits in Threads beschrieben: Ein **Context Switch** bezeichnet den Vorgang der CPU, einen Prozess vorübergehend zu pausieren und einen anderen an dessen Stelle zu laden. Dies kostet Zeit, da die Speicherräume überschrieben und Programmcode gelesen und initialisiert werden muss. Da in dieser Zeit keine anderen Prozesse den Switch abbrechen können, steht ein Context Switch im direkten Widerspruch zum Ziel der Response-Time.

Weil Context-Switches eben Zeit kosten, sollten sie nicht zu oft durchgeführt werden um unnötigen Overhead zu vermeiden.

Um effizient und nicht unnötig oft Prozesse zu tauschen, muss der Scheduler wissen, was die Prozesse machen wollen:

- **CPU-Bound:** Prozesse, welche primär die CPU brauchen und Dinge berechnen.
- **I/O-Bound:** Prozesse, welche primär mit der Hardware kommunizieren und die CPU nicht wirklich für große Berechnungen brauchen.

Kann der Scheduler einschätzen, welcher Prozess welche Anfragen stellen wird, kann er zuerst I/O Prozesse laden, die die Hardware beschäftigen, und dann während diese auf eine Antwort warten ("blocking") einen CPU-Bound Prozess laden, der zwischenzeitlich die CPU auslastet.

Das Problem ist, dass der Scheduler nicht in die Zukunft schauen kann, daher werden Prozesse **anhand ihrer bereits ausgeführten Befehle in die obigen Kategorien eingeteilt**. Idee dahinter: Wenn ein Prozess in der Vergangenheit viel CPU gebraucht hat, wird er es in Zukunft ebenso tun.

Context Switch Beispiel - Interrupt

Context Switches passieren zu verschiedenen Zeiten:

- Bei der Erstellung eines neuen Prozesses (Durch den Elternprozess gestartet)
- Bei Beendigung des aktuellen Prozesses

- Wenn ein Prozess in den "blocking" Zustand wechselt
- Wenn ein Interrupt / System Call auftritt

- + Auf der CPU wird gerade ein tolles Programm ausgeführt (ganz toll)
- + Plötzlich kommt ein IRQ (Interrupt Request), wird in den Cache gelegt
- + Interrupt Bit schaltet um, um IRQ anzuzeigen
- + Nach Beenden der aktuellen Instruktion prüft CPU das I-Bit
- + Sieht Interrupt, leitet Context Switch ein:
 - + Die derzeitigen Daten und der Instruction Pointer werden gesichert
 - + Richtet den Stack ein
 - + Lädt die ISR, welche den Interrupt behandelt
 - + Prozess ist abgeschlossen, CPU startet Scheduler
 - + Dieser wählt nächsten passenden Prozess
 - + Lädt die Speicherdaten, Stack, Instruction Pointer in die CPU
 - + Ende des Context Switch
- + CPU startet den Prozess

Schedulingalgorithmen

Key Takeaways

+ Algorithmus der entscheidet, welcher Prozess wann Zugriff auf den Prozessor bekommt

- Grobeinteilung in:
 - non-preemptive: Prozess wird bis zum Schluss ausgeführt, außer er blockiert
 - preemptive: Jeder Prozess bekommt maximale Prozessorzeit (Quantum), nach dessen Ablauf er spätestens abgelöst wird
- Diese Prinzipien sind Grundlage folgender Algorithmen

Schedulingalgorithmen haben die Aufgabe, möglichst passende Prozesse zu wählen und eine Balance zwischen den einzelnen Prozessen und der Menge an Context Switches herzustellen.

Grundlegende Schedulingarten

Es gibt verschiedene Arten des Scheduling, welche verschiedene Vor- und Nachteile liefern und für unterschiedliche Systeme geeignet sind. Generell lassen sie sich in 2 Unterkategorien einteilen:

Non-Preemptive Scheduling

Eine simple Art des Scheduling. Die CPU führt einen Prozess so lange aus, bis er entweder abgeschlossen ist, in den "blockieren" Modus wechselt oder bis er die CPU freiwillig hergibt

Preemptive Scheduling

Hier bekommt jeder Prozess ein Quantum, also einen maximalen Zeitraum, wenn er gestartet wird. Läuft dieser ab, kommt ein Interrupt oder blockiert der Prozess wird ein Context Switch ausgeführt.

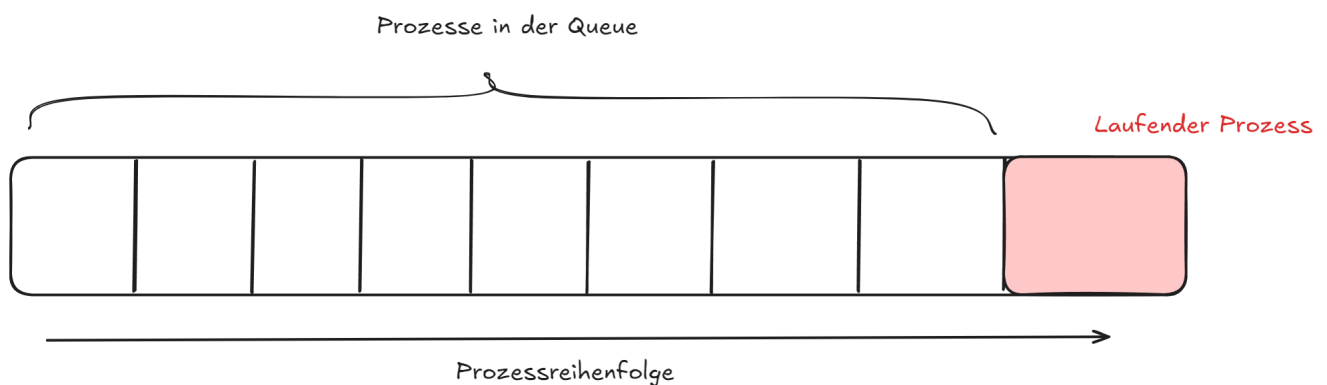
Im Folgenden werden einige Schedulingalgorithmen näher beleuchtet:

FIFO - First Come, First Serve

Key Takeaways

- Simpler Queue Algorithmus, alles kommt nacheinander, keine Unterbrechungen
- Sehr simpel, aber unresponsive und ungeeignet für Systeme mit Eingabe

Ein sehr simpler **non-preemptive** Schedulingalgorithmus. Er ist nichts weiter als eine Queue an Prozessen. Der erste Prozess der Queue, ist der erste Prozess der drankommt. Hierbei wird bei I/O Interrupts nicht unterbrochen, was FIFO für jegliche interaktive Systeme absolut ungeeignet macht.



Round Robin

Key Takeaways

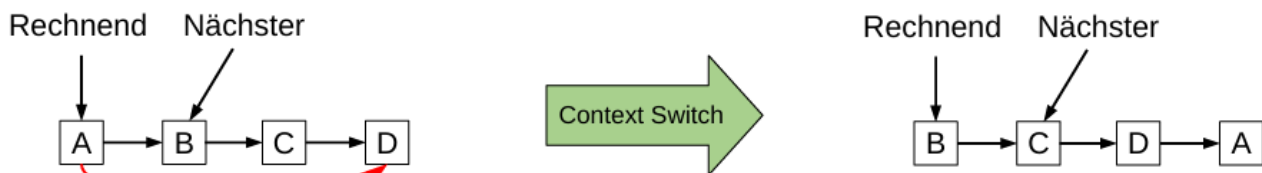
+preemptive Implementierung mit einer Liste

- Prozesse werden nacheinander in die CPU geladen, bis eine Bedingung passiert die einen Context-Switch erzwingt.
- Nächster Listeneintrag wird geladen, nicht fertige Prozesse hinten eingereiht
- Fairer Algo, aber keine Priorisierung ein Problem

Round Robin ist ein einfacher **preemptive** Schedulingalgorithmus. Hier werden Prozesse in einer Liste angelegt. Schält der rechnende Prozess in den blockiert Modus, oder läuft sein Quantum aus, wird er durch den nächsten Prozess in der Liste ersetzt. Der unterbrochene Prozess wird am Ende der Liste wieder angefügt.

Wichtige Punkte bei Round Robin:

- Jeder Prozess bekommt das selbe Quantum -> fair
- Dadurch bekommen aber auch unwichtigere Prozesse die selbe Zeit wie wichtigere -> Problematisch
- Einfach zu implementieren und tatsächlich gar nicht schlecht, weil er gut mit CPU Speed skaliert.
- Je nach Quantumgröße tritt eines der folgenden Probleme auf:
 - Kleines Quantum: Viele Context Switches, viel Overhead
 - Großes Quantum: Langes Warten anderer Prozesse, schlechtere Response Time



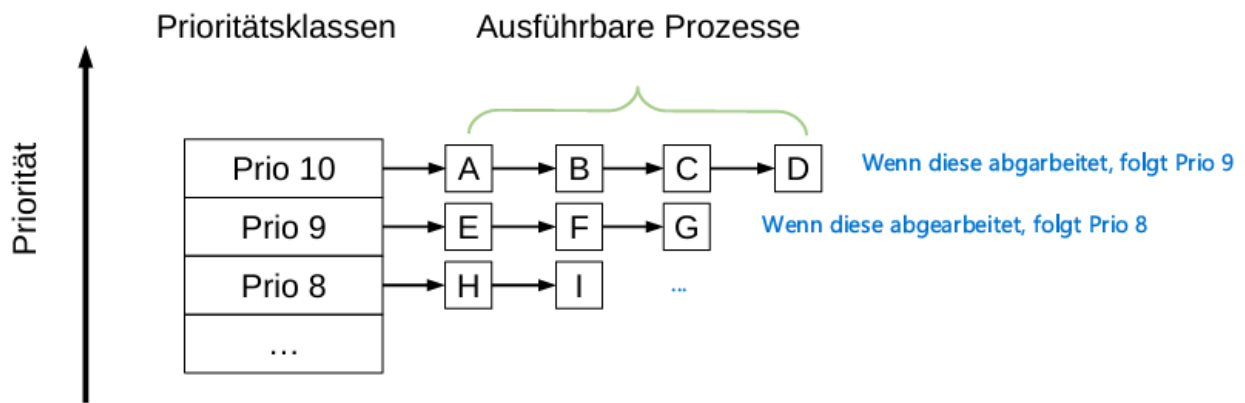
Prioritätsscheduling

Key Takeaways

- Preemptive Scheduler mit Prioritätsklassen
- Hohe Prio = Öfter dran
- Niedrige Prio = erst dran wenn es keinen mit höherer Prioklasse gibt
- Prinzipiell: IO Hochprior, CPU niedriprior
- Problem: Niedrige Prios sind sehr benachteiligt

Wie der Name vermuten lässt, erhalten Prozesse durch diesen Algorithmus eine **Priorität**. Je höher diese Priorität, desto öfter kommen die Prozesse dran. Dadurch soll erreicht werden, dass wichtige Prozesse schneller abgearbeitet werden als unwichtigere.

Prioritäten gibt es in Stufen. Das Scheduler geht stufenweise vor und beginnt von der höchsten Priorität. Prozesse mit niedrigerer Priorität können erst drankommen, wenn alle Prozesse mit höherer Priorität abgearbeitet wurden.



Die Prioritäten können entweder **statisch** sein, also im Vorhinein festgelegt, oder **dynamisch** vom OS berechnet werden. Um dynamisch entscheiden zu können, muss das OS die verbrauchte Quantumzeit aufzeichnen und die Prozesse aktiv in I/O- und CPU-Bound einteilen.

Welche Eigenschaften bestimmen die Priorität?

- I/O Prozesse > CPU Prozesse, da sie CPU nicht lange blockieren (Balance)
- Browser und andere User-Prozesse > zB periodischer E-Mail Check
- Multisusersysteme: zB Dekan-Prozesse > Studentprozesse

Beispiel

Annahme: Quantum = 100ms

Ein Prozess wird auf der CPU ausgeführt und verbraucht sein Quantum

Der Scheduler sieht, dass der Prozess 10ms für I/O und 90ms CPU gebraucht hat

Daher teilt er den Prozess als CPU Prozess ein -> bekommt niedrige Prio (es gilt: CPU > I/O ? CPU : I/O)

CPU-Bound Prio: $\text{Quantumzeit} / \text{Verbrauch} = 100\text{ms} / 90\text{ms} = 1.1$

I/O-Bound Prio: $\text{Quantumzeit} / \text{Verbrauch} = 100\text{ms} / 10\text{ms} = 10$

Hier würde also die CPU Priorität von 1.1 gewählt.

Innerhalb der Prioritätsklassen wird Round Robin verwendet. Das **große Problem** ist, dass niedrig priore Prozesse nur drankommen können, wenn es keine wichtigeren gibt, was zu sehr sehr langen Wartezeiten führen kann.

Multilevel Feedback Queues (MLFQs)

Key Takeaways

- Gleiche Logik wie Prioritätsscheduler, mit Zusatz:
- Je kleiner die Prio, desto mehr Zeit pro Aufruf auf der CPU

- Dadurch sollen hohe oft dran sein und kurz und niedrige selten aber länger
- insgesamt besseres balancing

Das Problem der verhungerten Niedrigprioritätsprozesse soll durch Multilevel Feedback Queues gelöst werden. Die Idee ist die selbe wie die des Prioritätsschedulings, mit einer Erweiterung:

Prozesse mit niedrigeren Prioritäten bekommen mehr Quanten

Das führt zu folgender Balance:

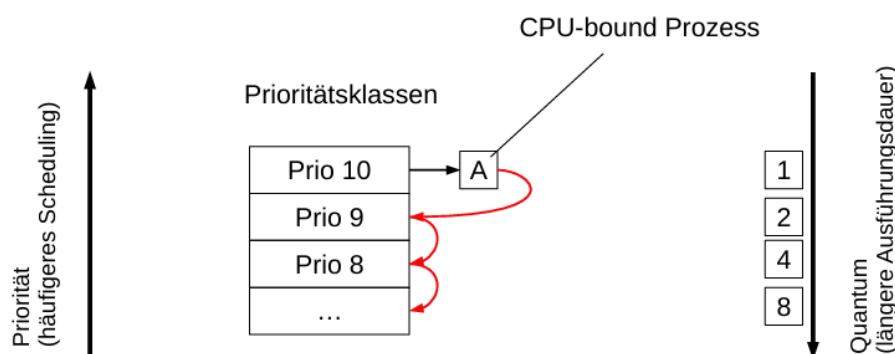
- Hohe Prio: Kommt oft dran, aber immer nur kurz
- Niedrige Prio: Kommt selten dran, dafür aber lange -> Wenig Context Switches

Die Hoffnung ist, niedrigpriori Prozesse in einem Stück abzuarbeiten und die Response Time der hochpriori Prozesse zu erhalten, da sie weiterhin oft drankommen.

Nach jedem Mal drankommen wird ein Prozess neu evaluiert und in eine neue Prioritätsklasse gesteckt. Das soll die Klassen korrekt und aktuell halten

Sowohl für IO und CPU Bound passend weil die einen wollen oft und dafür kurz und die anderen lang

MLFQ - Beispiel



- Beispiel: Prozess A benötigt 20 Quanten und kein I/O
 - Startet in Prioritätsklasse 10 und wird unterbrochen → kann nicht fertig rechnen
 - A wird daher nach unten gereiht (CPU-bound)
 - A wird das nächste Mal in Prioritätsklasse 9 gestartet
 - A bekommt daher weniger häufig die CPU
 - A bekommt dafür die CPU für eine längere Zeit
- Bsp: Was würde passieren wenn A plötzlich beginnt I/O Operationen durchzuführen? hochstufung -> weniger Quantum, dafür öfter
- Bsp: Vergleich Round Robin: 5 vs. 20 Context Switches

Implementierungen

Über die Zeit wurden verschiedene Scheduler in verschiedenen Betriebssystemen eingesetzt. Heutzutage werden oft erweiterte Formen der MLFQ verwendet. Früher wurden

von non-preemptives über Round Robin, quasi alles mal probiert.

(Hier gibt es folien dazu, halte ich aber für irrelevant)

Completely Fair Scheduler

Ein Linux-Scheduler, welcher auf MLFQ aufbaut: Zusätzlich zu den Standardfeatures besitzt er eine Maximum Excecution time, also ein Quantum, welches laufend wächst wenn der Prozess gerade nicht arbeiten darf. Je niedriger die Prio, desto schneller wächst sein Zeitrahmen.

EEVDF

Ähnlich zu Completely fair, halt ich auch für eher irrelevant